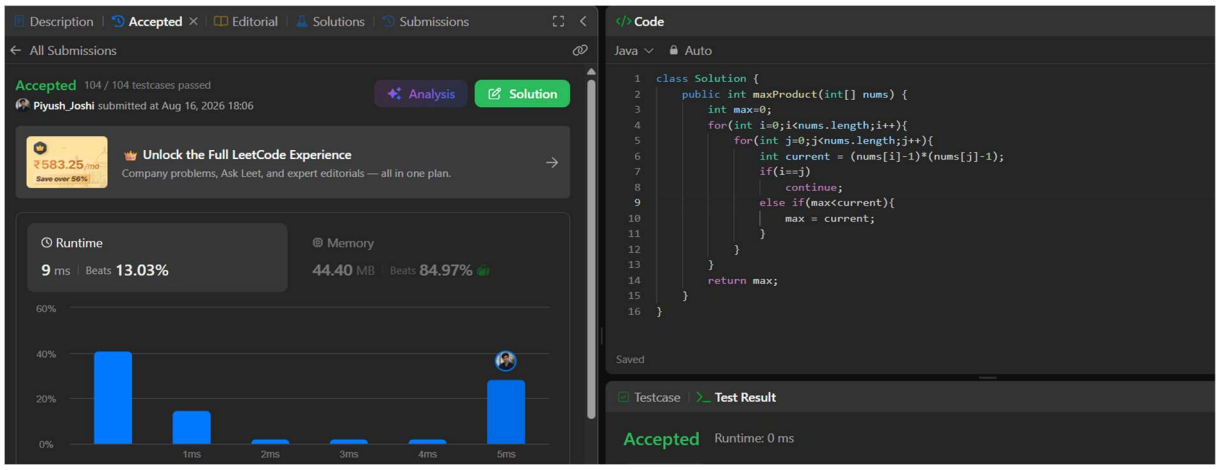
Coding ASSIGNMENT 1

Name	**Piyush Joshi**
Section	**FB**
Class Roll no.	**46**
University Roll no.	**12515001110**

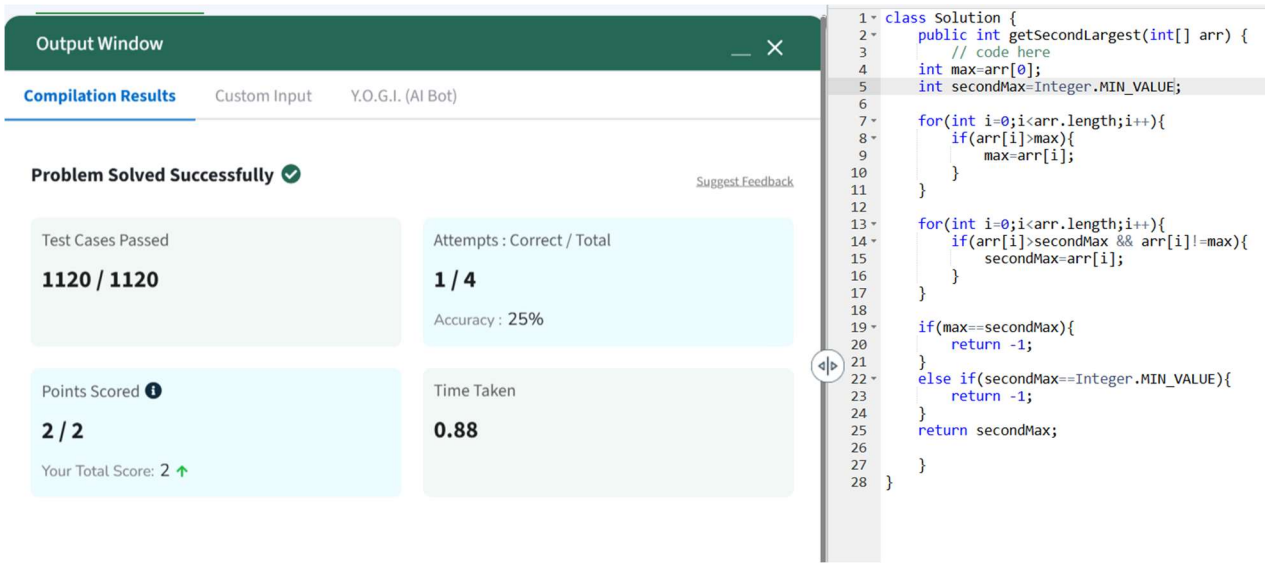
1. Maximum Product of Two Elements in an Array

```java
class Solution {
    public int maxProduct(int[] nums) {
        int max=0;
        for(int i=0;i<nums.length;i++){
            for(int j=0;j<nums.length;j++){
                int current = (nums[i]-1)*(nums[j]-1);
                if(i==j)
                    continue;
                else if(max<current){
                    max = current;
                }
            }
        }
        return max;
    }
}
```



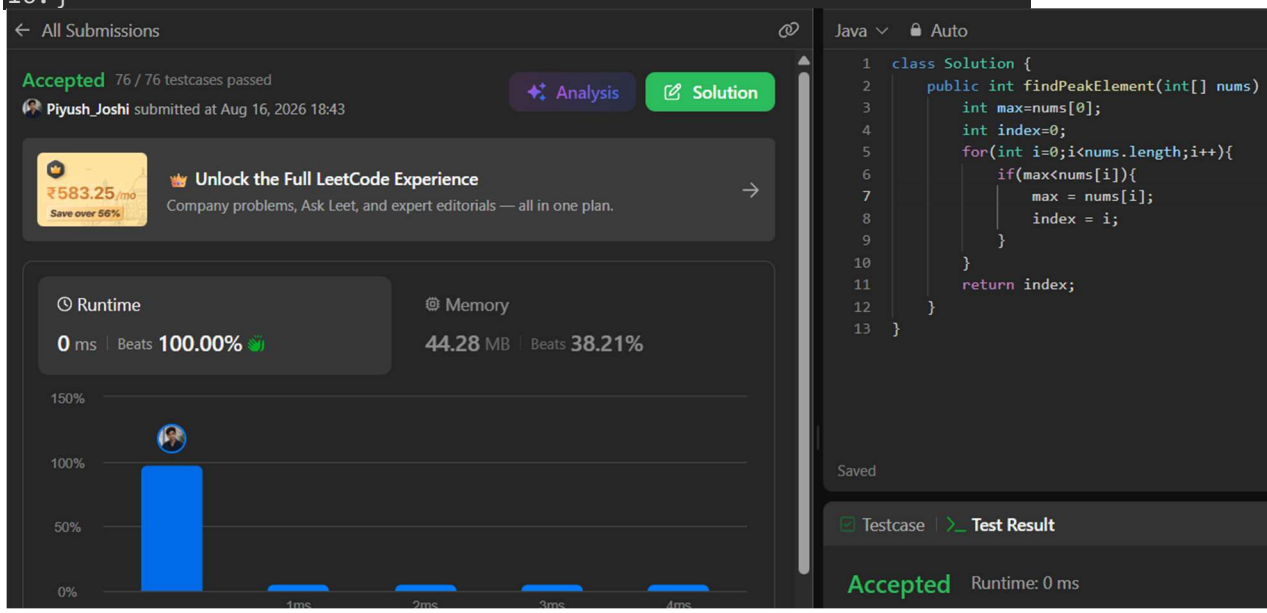
2. Second-Largest

```
class Solution {
   public int getSecondLargest(int[] arr) {
   int max=arr[0];
   int secondMax=Integer.MIN_VALUE;
   for(int i=0;i<arr.length;i++){
      if(arr[i]>max){
         max=arr[i];
 }
   }
   for(int i=0;i<arr.length;i++){
      if(arr[i]>secondMax && arr[i]!=max){
         secondMax=arr[i];
      }
   }
   if(max==secondMax){
      return -1;
   }
   else if(secondMax==Integer.MIN_VALUE){
      return -1;
   }
   return secondMax;
}}
```



3. Find Peak Element

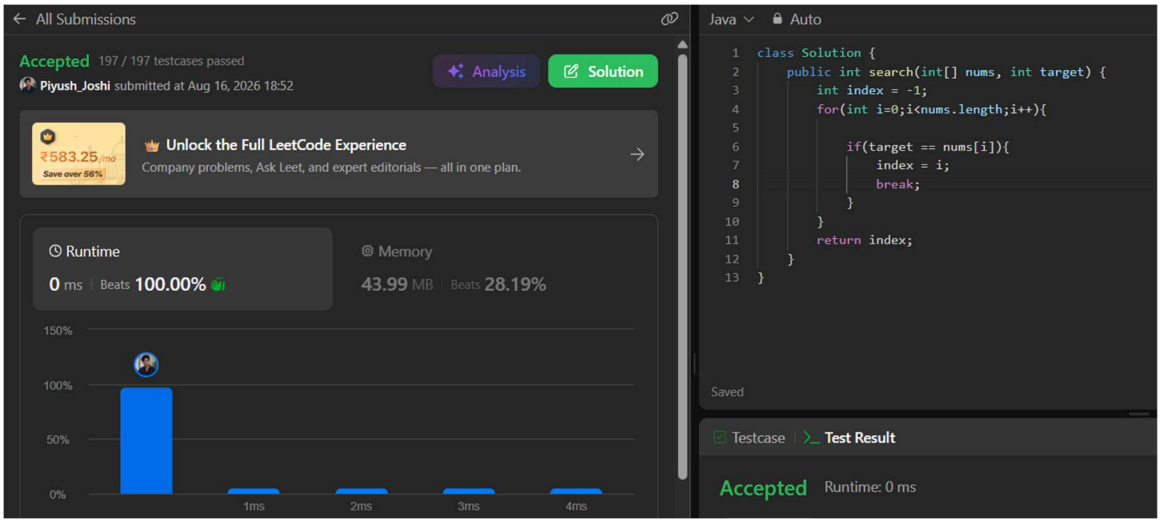
```java
class Solution {
    public int findPeakElement(int[] nums) {
        int max=nums[0];
        int index=0;
        for(int i=0;i<nums.length;i++){
            if(max<nums[i]){
                max = nums[i];
                index = i;
            }
        }
        return index;
    }
}
```



4. Search in Rotated Sorted Array

```java
class Solution {
    public int search(int[] nums, int target) {
        int index = -1;
        for(int i=0;i<nums.length;i++){

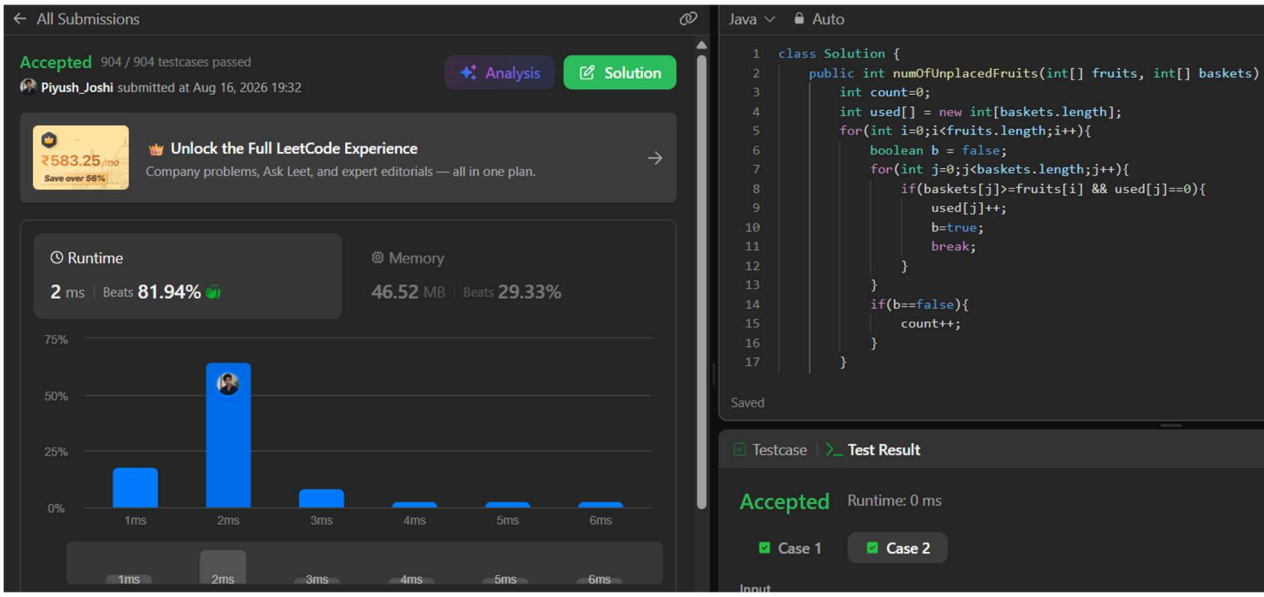
            if(target == nums[i]){
                index = i;
                break;
            }
        }
        return index;
    }
}
```



5. Fruits Into Baskets II

```java
class Solution {
    public int numOfUnplacedFruits(int[] fruits, int[] baskets) {
        int count=0;
        int used[] = new int[baskets.length];
        for(int i=0;i<fruits.length;i++){
            boolean b = false;
            for(int j=0;j<baskets.length;j++){
                if(baskets[j]>=fruits[i] && used[j]==0){
                    used[j]++;
                    b=true;
                    break;
                }
            }
            if(b==false){
```

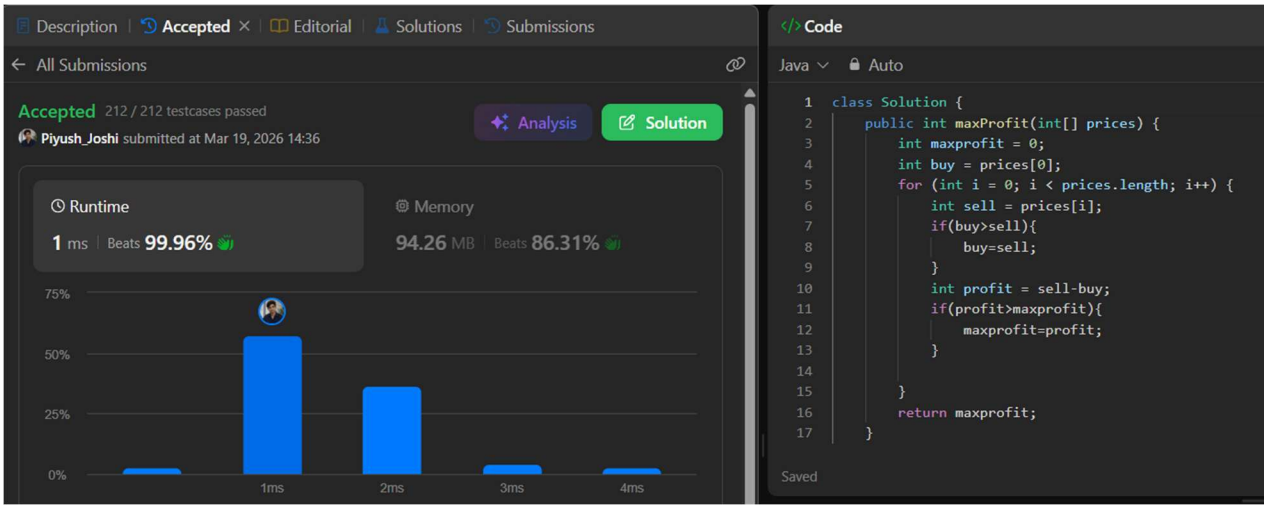
```
                count++;
            }
        }
        return count;
    }
}
```



6. Best Time to Buy and Sell Stock

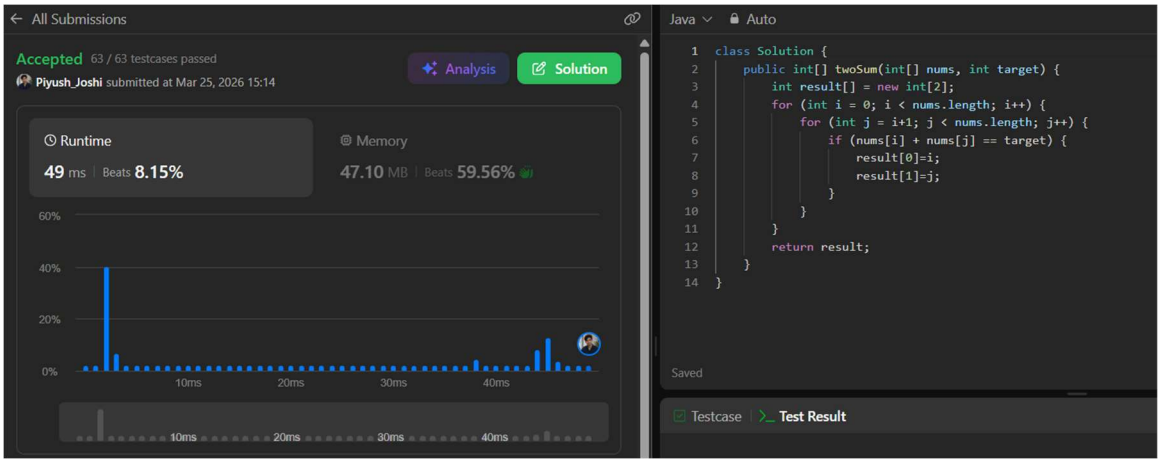
```
class Solution {
    public int maxProfit(int[] prices) {
        int maxprofit = 0;
        int buy = prices[0];
        for (int i = 0; i < prices.length; i++) {
            int sell = prices[i];
            if(buy>sell){
                buy=sell;
            }
            int profit = sell-buy;
            if(profit>maxprofit){
                maxprofit=profit;
            }

        }
        return maxprofit;
    }
}
```



7. Two Sum

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        int result[] = new int[2];
        for (int i = 0; i < nums.length; i++) {
            for (int j = i+1; j < nums.length; j++) {
                if (nums[i] + nums[j] == target) {
                    result[0]=i;
                    result[1]=j;
                }
            }
        }
        return result;
    }
}
```



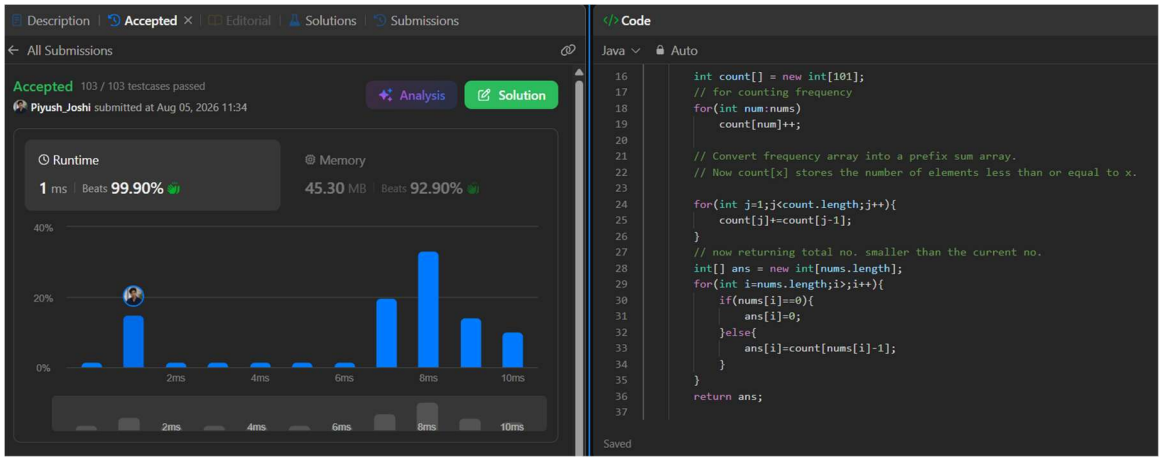
8. How Many Numbers Are Smaller Than the Current Number

```java
class Solution {
    public int[] smallerNumbersThanCurrent(int[] nums) {

        int count[] = new int[101];
        for(int num:nums)
            count[num]++;

        for(int j=1;j<count.length;j++){
            count[j]+=count[j-1];
        }
        int[] ans = new int[nums.length];
        for(int i=nums.length;i>;i++){
            if(nums[i]==0){
                ans[i]=0;
            }else{
                ans[i]=count[nums[i]-1];
            }
        }
        return ans;

    }
}
```

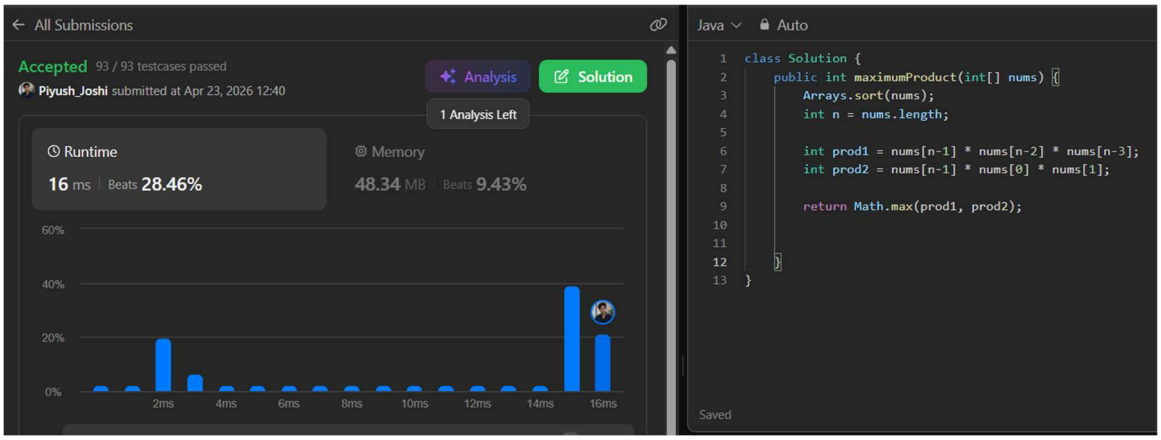


9. Maximum Product of Three Numbers

```java
class Solution {
    public int maximumProduct(int[] nums) {
        Arrays.sort(nums);
        int n = nums.length;

        int prod1 = nums[n-1] * nums[n-2] * nums[n-3];
        int prod2 = nums[n-1] * nums[0] * nums[1];

        return Math.max(prod1, prod2);
    }
}
```

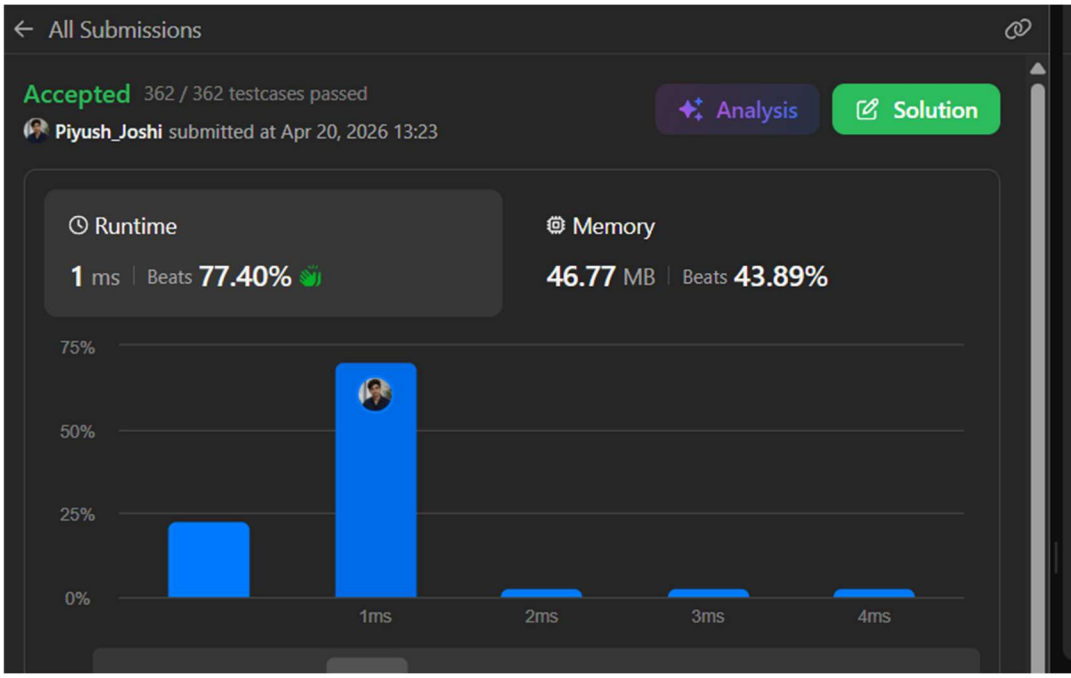


10.Remove Duplicates from Sorted Array

```java
class Solution {
    public int removeDuplicates(int[] nums) {
        if (nums.length == 0) return 0;

        int i = 0;
        for (int j = 1; j < nums.length; j++) {
            if (nums[j] != nums[i]) {
                i++;
                nums[i] = nums[j];
            }
        }

        return i + 1;
    }
}
```


All Submissions
Accepted 362 / 362 testcases passed
Piyush_Joshi submitted at Apr 20, 2026 13:23
Analysis
Solution
Runtime
1 ms | Beats 77.40%
Memory
46.77 MB | Beats 43.89%
75%
50%
25%
0%
1ms
2ms
3ms
4ms